Crear una base de datos PostgreSQL con Docker

Procedimiento sin utilizar Docker Compose

En esta práctica se desplegará un servidor PostgreSQL en un contenedor Docker, se creará una base de datos llamada empresa y se añadirá la tabla empleado. Los datos se almacenarán en un volumen independiente para que permanezcan aunque se elimine el contenedor.

Datos de la práctica

Imagen	postgres:17
Contenedor	postgresql
Volumen	postgres_datos
Base de datos	empresa
Usuario	admin
Contraseña de ejemplo	admin1234
Puerto	5432

Nota: admin1234 es una contraseña sencilla para una práctica local. En un entorno real debe utilizarse una contraseña robusta y evitar publicar el puerto del servidor innecesariamente.

1. Descargar la imagen de PostgreSQL

Descargamos la imagen oficial desde Docker Hub:

```
docker pull postgres:17
```

Podemos comprobar que se ha descargado:

```
docker images
```

2. Crear un volumen para los datos

El volumen permite conservar los archivos de la base de datos fuera del ciclo de vida del contenedor. Así, los datos no desaparecen al eliminarlo.

```
docker volume create postgres_datos
```

Para consultar los volúmenes existentes:

```
docker volume ls
```

3. Crear y arrancar el contenedor

Ejecutamos el siguiente comando:

```
docker run -d \
  --name postgresql \
  --restart unless-stopped \
  -e POSTGRES_USER=admin \
  -e POSTGRES_PASSWORD=admin1234 \
  -e POSTGRES_DB=empresa \
  -p 5432:5432 \
  -v postgres_datos:/var/lib/postgresql/data \
  postgres:17
```

Significado de los parámetros:

Parámetro	Descripción
-d	Ejecuta el contenedor en segundo plano.
--name postgresql	Asigna el nombre postgresql al contenedor.
--restart unless-stopped	Reinicia el contenedor automáticamente salvo que se haya parado expresamente.
POSTGRES_USER	Crea el usuario inicial admin.
POSTGRES_PASSWORD	Establece la contraseña inicial del usuario.
POSTGRES_DB	Crea inicialmente la base de datos empresa.
-p 5432:5432	Publica el puerto 5432 del contenedor en el equipo anfitrión.
-v ...	Monta el volumen persistente en el directorio de datos de PostgreSQL.
postgres:17	Indica la imagen y versión principal que se utilizarán.

Importante: las variables POSTGRES_USER, POSTGRES_PASSWORD y POSTGRES_DB se aplican durante la inicialización de un directorio de datos vacío. Cambiarlas después no modifica automáticamente una instalación ya inicializada.

4. Comprobar que el contenedor está funcionando

Para ver los contenedores en ejecución:

```
docker ps
```

Para consultar los mensajes de arranque:

```
docker logs postgresql
```

Cuando PostgreSQL esté preparado, los registros mostrarán un mensaje similar a “database system is ready to accept connections”.

5. Conectarse a PostgreSQL

Utilizamos el cliente psql que incluye la propia imagen:

```
docker exec -it postgresql psql -U admin -d empresa
```

El parámetro -U indica el usuario y -d la base de datos. Si la conexión es correcta, aparecerá el indicador empresa=#.

6. Crear la tabla empleado

Dentro de psql, ejecutamos la siguiente sentencia SQL:

```
CREATE TABLE empleado (
    numero         INTEGER PRIMARY KEY,
    nombre         VARCHAR(20),
    apellido       VARCHAR(20),
    foto           VARCHAR(20),
    sueldo         NUMERIC(6,2),
    sueldoMaximo   NUMERIC(6,2),
    fechaAlta      DATE
);
```

Para comprobar que la tabla existe:

```
\\dt
```

Para consultar su estructura:

```
\\d empleado
```

7. Insertar registros de prueba

Insertamos tres empleados:

```
INSERT INTO empleado VALUES
(1, 'Manuel', 'Garcia', 'manuel.jpg', 1800.50, 2500.00, '2024-01-15'),
(2, 'Ana', 'Lopez', 'ana.jpg', 2100.00, 2800.00, '2023-06-10'),
(3, 'Pedro', 'Martin', 'pedro.jpg', 1950.75, 2600.00, '2022-09-01');
```

Consultamos todos los registros:

```
SELECT * FROM empleado;
```

También podemos ordenar los empleados por sueldo:

```
SELECT nombre, apellido, sueldo
FROM empleado
ORDER BY sueldo DESC;
```

Para salir del cliente psql:

```
\\q
```

8. Parar y arrancar el contenedor

Detenemos el contenedor:

```
docker stop postgresql
```

Volvemos a arrancarlo:

```
docker start postgresql
```

Comprobamos su estado y nos conectamos de nuevo:

```
docker ps
docker exec -it postgresql psql -U admin -d empresa
```

Dentro de psql, ejecutamos:

```
SELECT * FROM empleado;
```

Los registros siguen disponibles porque se guardan en el volumen postgres_datos.

9. Eliminar el contenedor

Podemos eliminar el contenedor sin borrar el volumen:

```
docker stop postgresql
docker rm postgresql
```

El volumen postgres_datos permanece independiente. Para crear de nuevo el contenedor utilizando los mismos datos, ejecutamos otra vez:

```
docker run -d \
  --name postgresql \
  --restart unless-stopped \
  -e POSTGRES_USER=admin \
  -e POSTGRES_PASSWORD=admin1234 \
  -e POSTGRES_DB=empresa \
  -p 5432:5432 \
  -v postgres_datos:/var/lib/postgresql/data \
  postgres:17
```

Al reutilizar el volumen, PostgreSQL encontrará los datos ya inicializados. Las variables de entorno no volverán a crear ni modificar el usuario o la base de datos.

10. Eliminar también los datos

Si queremos borrar definitivamente la base de datos y todos sus archivos, debemos eliminar el contenedor y después el volumen:

```
docker stop postgresql
docker rm postgresql
docker volume rm postgres_datos
```

¡Atención! El último comando elimina permanentemente los datos guardados en el volumen. No debe ejecutarse si queremos conservar la base de datos.

11. Conectarse desde NetBeans o DBeaver

Como hemos publicado el puerto 5432, las aplicaciones instaladas en el equipo anfitrión pueden conectarse al servidor PostgreSQL.

Parámetro	Valor
Servidor	localhost
Puerto	5432
Base de datos	empresa
Usuario	admin
Contraseña	admin1234
Driver JDBC	PostgreSQL JDBC Driver

La URL de conexión JDBC sería:

```
String url = "jdbc:postgresql://localhost:5432/empresa";

Connection conexion = DriverManager.getConnection(url, "admin", "admin1234");
```

Para utilizar JDBC desde un proyecto Java, hay que añadir el controlador JDBC de PostgreSQL a las dependencias del proyecto o a sus bibliotecas.

En JDBC moderno (JDBC 4), Class.forName(), ya no es obligatorio si el driver está correctamente incluido en el proyecto

```
Class.forName("org.postgresql.Driver");
```

Resumen de comandos

Operación	Comando
Descargar imagen	docker pull postgres:17
Crear volumen	docker volume create postgres_datos
Ver contenedores	docker ps
Ver registros	docker logs postgresql
Entrar en PostgreSQL	docker exec -it postgresql psql -U admin -d empresa
Parar contenedor	docker stop postgresql
Arrancar contenedor	docker start postgresql
Eliminar contenedor	docker rm -f postgresql
Ver volúmenes	docker volume ls
Eliminar volumen	docker volume rm postgres_datos